AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
i-CREDIT HOURS ENGINEERING PROGRAMS

Tiny Telemetry Protocol: Efficient Periodic Telemetry over UDP with Optional Batching

CSE361: Computer Networking
Submitted to: Prof/Ayman Bahaa, Dr/Kareem Emara

Submitted by:

1. Abdelrahman Adel 23P0144
2. Monzer Mahmoud 23P0122
3. Omar Ahmed Abouraia 23P0100
4. Hossam Osama 23P0010
5. Zeina Reda 23P0181
6. Logine Mohamed 23P0187

1. Introduction

TinyTelemetry is a lightweight, UDP-based telemetry protocol designed for periodic reporting of sensor readings from embedded devices to a central collector. The protocol prioritizes minimal overhead, predictable timing, and robustness under lossy conditions. It is particularly suited for IoT sensor deployments where devices report data at fixed intervals and cannot afford the overhead or complexity of connection-oriented protocols.

Traditional TCP-based telemetry approaches introduce undesirable properties for this use case: connection establishment delays, retransmissions that introduce jitter, head-of-line blocking, and relatively large headers. TinyTelemetry instead adopts a compact fixed header, stateless communication, and optional batching to reduce byte overhead and processing load.

Assumptions and Constraints:

- Transport: UDP (no retransmissions)
- Max packet size: 200 bytes
- Expected network loss: 5–15%
- Reporting intervals: 1s, 10s, 30s
- 16-bit DeviceID
- Sequence-number–based detection of duplicates, gaps, and reordering

1.1 Justification: Difference from MQTT-SN

TinyTelemetry differs from MQTT-SN in several ways:

1. Stateless Design: MQTT-SN requires topic registration and a gateway.
   TinyTelemetry is stateless: no sessions, no ACKs, no broker.
2. Direct Push vs Publish/Subscribe: MQTT-SN uses topics and broker routing.
   TinyTelemetry pushes telemetry directly to the collector.
3. Deterministic Timing: MQTT-SN uses QoS-based retransmissions that introduce jitter. TinyTelemetry avoids retransmission entirely.
4. Header Size: MQTT-SN headers vary between 7–9+ bytes.
   TinyTelemetry uses a fixed 12-byte header.
5. Purpose-Built: MQTT-SN is general-purpose;
   TinyTelemetry is optimized for periodic telemetry.

2. Protocol Architecture

TinyTelemetry defines a unidirectional client → server model over UDP with no handshake and no session state.

Client FSM:

START → INIT → REPORTING(DATA/HB) → END

3. Message Format

Header (struct.pack): !B H I I B

• Version (4 bits)

• MsgType (4 bits)

• DeviceID (16 bits)

• SeqNum (32 bits)

• Timestamp (32 bits)

• Flags (8 bits: batch=1, checksum=4)

Payload Formats:

Single Reading: [Type_Unit][Float][Checksum?]

Batch: [Count] + Count × ([Type_Unit][Float][Checksum?])

Type_Unit Byte: High nibble: sensor type **,** Low nibble: unit

RFC Batching: Batch sizes typically 3–8, independent of interval.

Experimental Batching (Phase 2):

1s → 1 reading

10s → 10 readings

30s → 30 readings

4. Communication Procedures

- INIT: Client sends INIT(seq=0). Server resets device state.
- DATA: Client sends readings periodically. Server parses, checks type/unit, logs stats.
- HEARTBEAT: Sent every 10 packets.
- Error Handling: Duplicate, gap, and reorder detection is sequence-based.
- Shutdown: Client stops; server prints summary.

5. Reliability & Performance Features

- Sequence-Based Detection: Identifies duplicates, gaps, and reordered packets.
- Optional Checksum: 1-byte modulo checksum increases robustness.
- Batching: Amortizes header overhead.
- No Retransmissions: Keeps timing deterministic.
- Interval-Based Batching (Evaluation Only): Used for performance studies but not part of RFC.
- Handling Lost Packets via Moving-Average Estimation (Requirement-4 Alternative): Although TinyTelemetry does not retransmit lost packets, the collector may optionally apply a moving-average estimator to approximate missing values.
  Algorithm: When a seq gap occurs: estimated_value =mean(last_k_values)
  This is for analytics only; it does not modify the protocol or require retransmission.

6. Experimental Evaluation Plan

Metrics Collected

The following metrics were collected by the collector and logged to CSV for ofline analysis:

- bytes_per_report: average number of bytes transmitted per sensor reading
- duplicate_rate: ratio of duplicate packets to total received packets
- sequence gaps: inferred packet loss based on sequence number gaps
- CPU time per report: server-side processing time per packet

These metrics allow evaluation of both protocol efficiency and behavior under loss.

Reporting Interval Experiments

To evaluate the impact of reporting interval and batching on protocol efficiency, the client was executed with batching enabled at three different reporting intervals:

```
python client.py --interval 1  --batch
python client.py --interval 10 --batch
python client.py --interval 30 --batch
```

For evaluation purposes only, the client employed an interval-dependent batching policy:

- 1 second → 1 reading per packet
- 10 seconds → 10 readings per packet
- 30 seconds → 30 readings per packet

This policy is not part of the protocol specification, but was used to study how batching amortizes header overhead.

Results and Interpretation

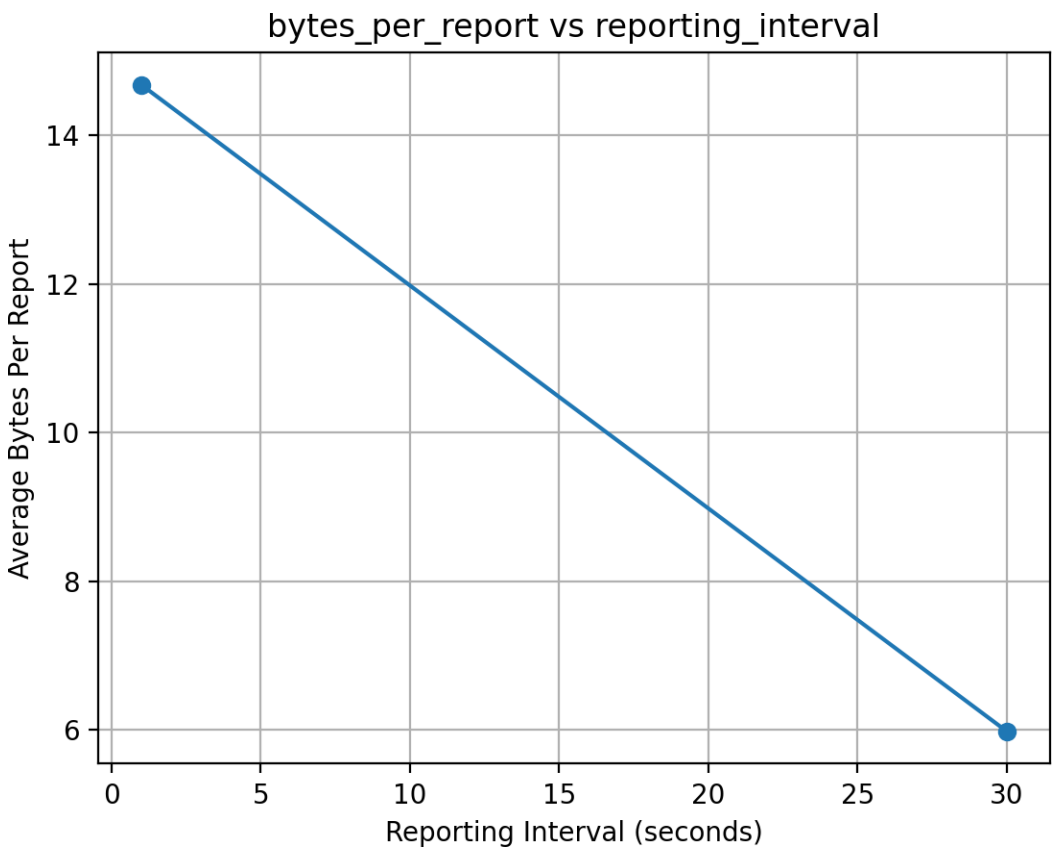

Figure 1: bytes_per_report vs reporting_interval

The average bytes per report decrease significantly as the reporting interval increases. This behavior is expected because:

- The TinyTelemetry header has a fixed size (12 bytes)
- Larger reporting intervals allow more readings to be batched into a single packet
- Header overhead is therefore amortized across more readings

This demonstrates that TinyTelemetry efficiently scales for low-frequency reporting scenarios and validates the effectiveness of application-layer batching.

Packet Loss Experiments

To evaluate protocol behavior under packet loss, loss was injected using Linux netem (via WSL) while keeping the reporting interval fixed at 1 second:

sudo tc qdisc add dev eth0 root netem loss 5%

python client.py --interval 1

sudo tc qdisc add dev eth0 root netem loss 15%

python client.py --interval 1

sudo tc qdisc del dev eth0 root

The baseline case (0% loss) was collected without netem.

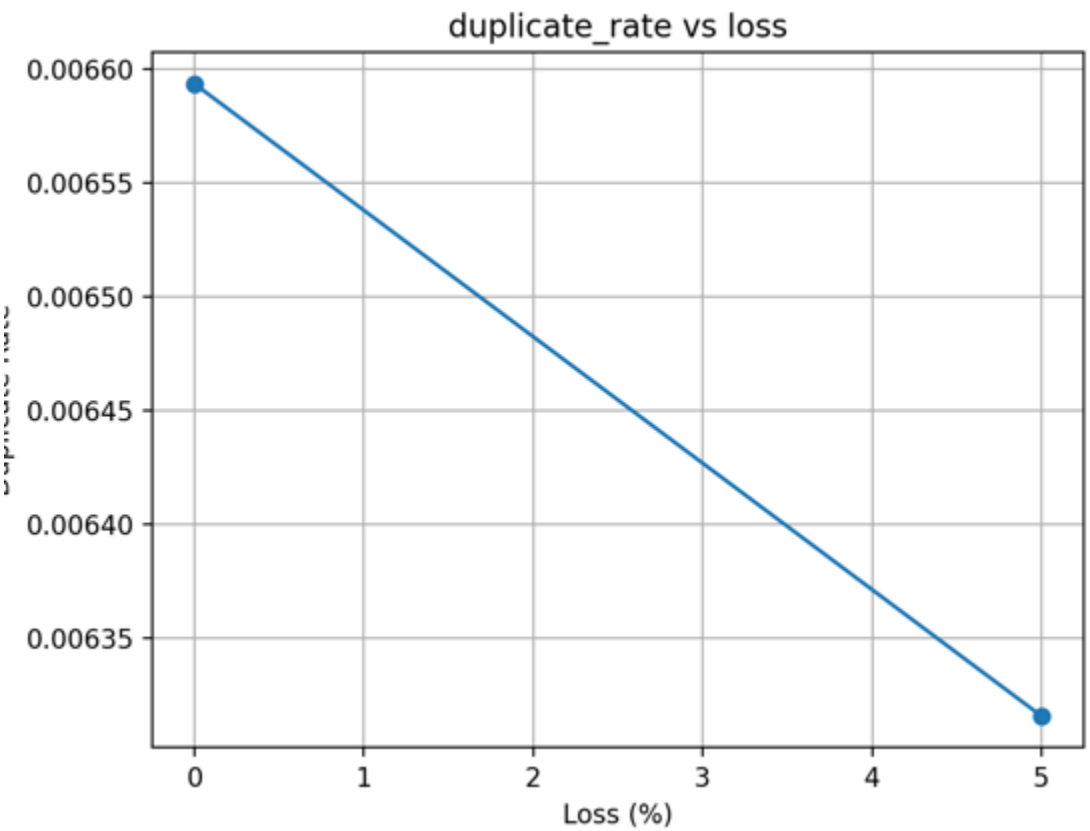

Figure 2: duplicate_rate vs loss

The measured duplicate rate decreases slightly as packet loss increases.

Although this may appear counterintuitive, this behavior is consistent with the TinyTelemetry design:

- The protocol uses UDP with no retransmissions
- Lost packets are not resent by the client
- As loss increases, the total number of packets received by the server decreases
- Fewer received packets result in fewer opportunities for duplicate observations

Thus, the observed decrease in duplicate rate reflects the protocol's loss-tolerant, non-retransmitting design, rather than an error in implementation.

7. Example Use Case Walkthrough

Scenario: Device 999 Starts Reporting Temperature Readings

The following steps illustrate a typical session from the device's initiation to normal data reporting, including timestamps, sequence numbers, and communication.

1. **Client Sends INIT Message (seq=0)**
    - **Purpose:** To initialize or reset the device's state on the server.
    - **Details:** Sequence number starts from 0, and the server will reset per-device tracking for lost, duplicate, or reordered packets.

Example:

INIT(seq=0) → Server resets per-device counters.

2. **Client Sends Data Packets (seq=1 to 10)**
    - **Purpose:** The device sends telemetry readings, one per second.
    - **Details:** Each DATA packet includes a sequence number (seq), a timestamp, and the sensor value (e.g., temperature).

Example:

DATA(seq=1), value=22.3°C → Server logs device ID, seq, timestamp, and reading.

DATA(seq=2), value=22.4°C → Server logs next value.

…

DATA(seq=10), value=22.5°C → Server logs the last data value in the current session.

3. **Client Sends HEARTBEAT (seq=10)**
    - **Purpose:** To notify the server that the device is still active and functioning.
    - **Details:** A **HEARTBEAT** packet is sent after every 10 data packets. It has no payload and signals that the client is still alive.

Example:

HEARTBEAT(seq=10) → Server receives the heartbeat and updates internal state.

CSV Log Example

The server logs every packet received, noting the sequence number, timestamp, flags (duplicate/gap), and computation of **bytes per report**.

For example, when the server receives **DATA** packets, it logs:

999, 1, 1765287101, 1765287101.115, FALSE, FALSE, 0.041ms, 17.0

999, 2, 1765287101, 1765287102.121, FALSE, FALSE, 0.037ms, 17.0

999, 3, 1765287101, 1765287103.128, FALSE, FALSE, 0.043ms, 17.0

…

999, 10, 1765287101, 1765287110.145, FALSE, FALSE, 0.052ms, 17.0

Server Statistics Summary on Shutdown

When the server is stopped (e.g., with **Ctrl + C**), it will output a summary of **packets received**, including information on duplicates, packet loss, reordering, and resets:

--- Final Server Stats Summary ---

Device 999: received=60, lost=0, loss%=0.0, dups=0, reorders=0, resets=1

--- End Summary ---

Here's the interpretation:

- **received=60**: 60 packets were received from device 999.
- **lost=0**: No packets were lost.
- **loss%=0.0**: Loss percentage is zero.
- **dups=0**: No duplicate packets were detected.
- **reorders=0**: No packets were reordered.
- **resets=1**: The server initiated one reset.